

Old vs New Input & Output *Signals*

Old Way – @Input()

@Input() user!: User;

- ✗ Not reactive
- ✗ No dependency tracking
- ✗ Change detection runs more

New Way – Input Signal

```
user = input<User>();
```

Usage:

```
{{ user()?.name }}
```

- ✓ Reactive input
- ✓ Works with computed
- ✓ Cleaner code

Real-World Example

- 🧠 Parent changes user
 - ➡ Child auto updates
 - ➡ No manual logic

```
isAdmin = computed(() =>  
  this.user()?.role === 'ADMIN'  
);
```

Perfect for dashboards & permissions

Old Way – @Output()

```
@Output() save = new  
EventEmitter<void>();
```

```
this.save.emit();
```

✗ Not signal-based

✗ Legacy pattern

New Way – Output Signal

```
save = output<void>();
```

```
this.save.emit();
```

- ✓ Signal-friendly
- ✓ Cleaner API
- ✓ Future-ready Angular

Why Signals Are Faster



Angular knows:

- Who uses the signal
- Where to update



No full page re-check



Only required UI updates

Interview Ready Summary

- ✓ Signals = reactive UI state
- ✓ RxJS = async streams
- ✓ Input/Output signals = modern Angular
- ✓ Better performance than old CD

 One-Line Answer

“Signals bring fine-grained reactivity to Angular.”

Final Thought

Angular is moving towards:

- ⚡ Signals
- ⚡ Standalone Components
- ⚡ Zone-less future

📌 Save this carousel

📌 Share with Angular devs

**Follow
for more
great tips**
